RAY – AI Automation Engineer Case Study

Benjamin Kariuki · benjaminleyian50@gmail.com · November 2026

Code repository	github.com/benjaminkariuki/ray-ai-automation-case-study (both tasks)
Requirement checklist	REQUIREMENTS_CHECK.md: every line of the brief, how it's met, where to verify it

Each task starts with its own links, then what was built, how every requirement is met, and how to verify it.

Task 1 – Executive Assistant Job Outreach Automation

Google Sheet (dashboard)	docs.google.com/spreadsheets/d/1GwAt8R3_O-h6ivZ_CT_8yD6GMIdybyf2-YgscTQ9hRI Anyone with the link can view and edit
Run it now (optional)	ray-run-now.run-now.workers.dev A complementary extra for manual runs. The automation already runs by itself every day (the default); this page only lets you start an additional run on demand, without a GitHub account.
Run history	GitHub Actions: the automatic daily runs, plus any manual runs
README	task1-job-outreach/README.md
Sample output	sample-output/: CSV of each Sheet tab, plus `latest/` (console log, JSON, CSV of the newest run)

What I built

A **Python pipeline**, run **automatically every day by GitHub Actions** (the default; no one needs to start it), so no server of our own is needed. Each run:

- **Searches three job boards** (Himalayas, RemoteOK and Jobicy) through their official public APIs. LinkedIn is not used.
- **Checks each new posting with an LLM**: is it remote, English-speaking and relevant to an Executive or Admin Assistant? A job must pass all three to be kept.
- **Writes a personalised outreach email** for each match.
- **Fills a colour-coded Google Sheet** that sales reps use as their dashboard.

Search terms, filters, AI models and prompts are all edited **in the Sheet**, so tone can be tested without touching code.

```
Automatic: daily ~06:07 UTC (+ backups)   ·   Optional manual: "Run now" page / "Run workflow"  →  python -m job_outreach
  1  Read Settings + Prompts + already-processed URLs from the Google Sheet (bad values fall back to defaults)
  2  Fetch: Himalayas · RemoteOK · Jobicy   →  every posting gets title, company, location, description, URL, languages
  3  Rule filter: title keywords · excluded companies/agencies · max age · duplicates        (no AI cost)
  4  AI check per new job  →  JSON {remote, English, relevant, agency, languages, reason}
  5  AI outreach email for each job that passes all three  (quality-checked; regenerated if not clean)
  6  Write Leads + Rejected, then the Run Log row  ·  save JSON / CSV / console log  ·  verify the live Sheet
  AI: OpenRouter (primary)  ──fails or hits its limit──►  Cloudflare Workers AI (backup)  ──both fail──►  retried next run
```

How to verify Task 1 in 5 minutes

1. Open the **Sheet**. In **Leads**, compare 2–3 messages with their Job Posting URL; each is specific to its posting.
2. **Rejected** shows why each job was dropped (the failed check in red). **Prompts** and **Settings** show what's configurable. **Run Log** shows every run.

3. **Scheduling:** the **Run Log** and **Run history** show the automatic daily runs (Trigger "Schedule …", in green). Optional: to see a run happen now, open **Run it now** and press the button. About 3–6 minutes later a new Run Log row appears with Trigger "Run now (public link)". You can first set `Settings → outreach_prompt` to `v2_direct`.
4. In the repo, open `task1-job-outreach/sample-output/` for the CSV/JSON output and the newest run's console log.
5. Optional: run `python task1-job-outreach/scripts/verify_sheet.py`; expect 13/13 PASS.

Requirements, as written in the brief

Brief	How it's met
✓ Compliant, non-LinkedIn job board	3 official public APIs, with no HTML scraping. Each row keeps the original URL and source name (the boards ask for a link back), and requests are paced.
✓ Collect title, company, location, description, URL and language requirements for each posting	All six fields for **every** scraped posting. Language requirements come from rules for every posting and are refined by the AI for checked jobs.
✓ Store in a structured format	Each run saves **JSON** (every posting and AI decision), **CSV** (all postings, leads, rejections) and a console **log**, plus the Google Sheet.
✓ Remote + English + relevant; keep only postings meeting all three	The LLM returns strict JSON with a reason. Only all-three passes become leads; the rest go to the **Rejected** tab with Yes/No per check. Staffing and VA agencies (competitors) are also excluded, which is configurable.
✓ Genuinely personalised message per posting	Each message cites duties, tools or details from that posting. Facts about us come only from the Sheet's "About us" setting, so nothing is invented.
✓ Easily interchangeable prompt	Prompts live in the Sheet's **Prompts** tab, and changing `Settings → outreach_prompt` switches them. Each lead records **Prompt Used**; both `v1_warm` and `v2_direct` were tested live.
✓ Google Sheet columns	The 6 required columns come first, in order, then Language, Source, dates, Prompt Used, AI Model, **Status** (a dropdown) and Notes.
✓ Daily or weekly schedule	GitHub Actions, daily at about 06:07 UTC; one edit makes it weekly. GitHub's free scheduler can skip a slot, so backup slots every 3 hours plus an independent Cloudflare trigger make sure one run happens every day, and a failed run is retried up to 3 times. This automatic schedule is the default. As a complementary extra, anyone can also start an additional run manually from the public **Run now** page.
✓ README: install, run, filters and prompt, troubleshooting, sales rep guide	All covered, plus a 5-minute evaluator guide. A short guide is also built into the Sheet.

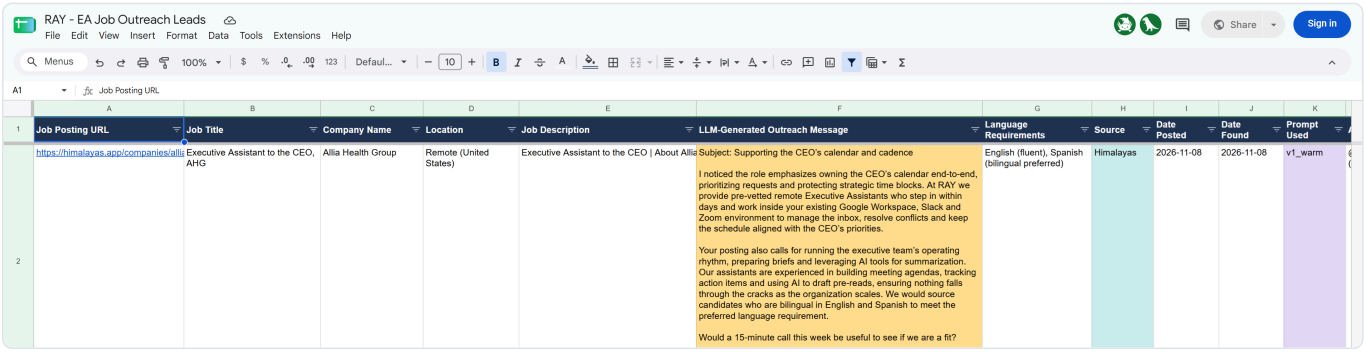
Deliverables

Deliverable	Where
Source code: scrape, validate, LLM, Sheets	`sources.py` · `ai.py` + `prompts/` · `ai.py` · `sheet.py` + `setup_sheet.py`, in `task1-job-outreach/`
Sample data output (CSV/JSON or console)	All three, in `sample-output/`, refreshed automatically after every run
Scheduling setup	`.github/workflows/job-outreach.yml`, documented in README section 5
README	`task1-job-outreach/README.md`
Populated Google Sheet	The link at the top of this task: leads, rejections with reasons, and run history

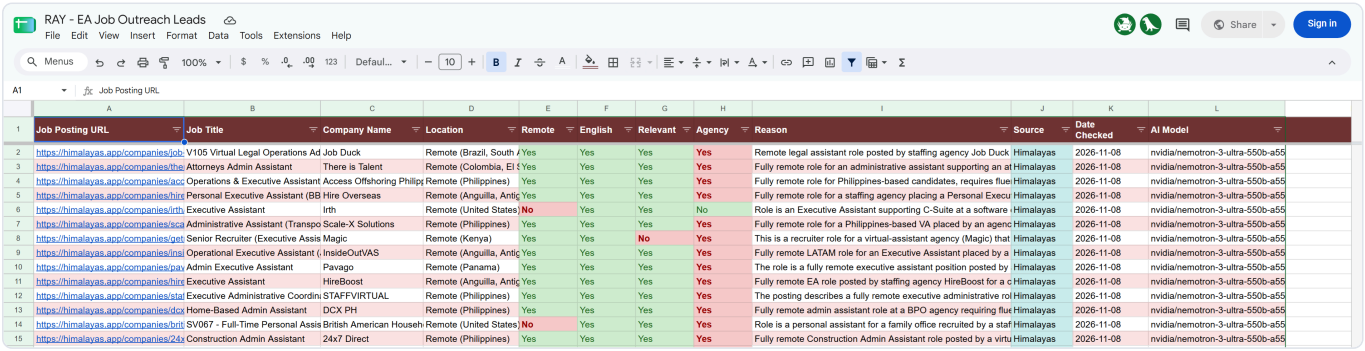
Results and testing

~345	39	37	13/13
postings fetched per run (3 boards)	leads written, each with a personalised email	rejections, each with a reason	live Sheet checks pass after every run

- **17 offline unit tests** (filtering, language detection, parsing, formatting, AI failover, quality checks, broken Settings) run before every run.
- `verify_sheet.py` checks the live Sheet after every run: columns, no blanks, no duplicates, sources, clean, personalised and unique messages, rejection logic, freshness. It needs no login, so anyone can run it.
- **Tested live:** OpenRouter's limit ran out mid-test and Cloudflare took over with 0 errors; the prompt was swapped from the Sheet; duplicates were skipped; dry run; the Run now page.
- **Problems found and fixed during testing:**
  - The model invented details (a language skill, tool expertise): prompts tightened, messages rewritten.
  - The backup model sometimes leaked its hidden reasoning into a message: replies are now cleaned and quality-checked, and the 2 affected messages were rewritten.
  - An incomplete AI answer caused a false rejection, and one job was marked "not English" only because English wasn't mentioned: both fixed.
  - GitHub skipped the first scheduled run: backup slots and the Cloudflare trigger were added.



The Leads tab: the six required columns first (the AI-written message highlighted in yellow), then language requirements, source, dates, prompt used, AI model and a Status dropdown for the sales rep. Rows alternate blue and white; the Source and Prompt Used columns are colour-coded.



The Rejected tab: every job the AI filtered out, with Yes/No for each check (the failed one in red) and the AI's reason.

Process note: I first built this in n8n on a self-hosted server. It worked, and its testing drove the fixes above. I then moved it to Python on GitHub Actions so nothing depends on a server staying alive during the evaluation.

Task 2 – Executive Travel Planner (Notion)

Notion page (public)	https://billowy-target-f39.notion.site/Executive-Travel-Planner-Benjamin-Kariuki-412d73ee564d807d82e8f40be8b6009c
README	task2-travel-planner/README.md

What I built

One Notion page, **"Executive Travel Planner – Benjamin Kariuki"**, published to the web (view-only). It holds **one Trips database** with **23 sample trips to Europe and the USA** (7 Done, 1 In-Progress, 15 Planned), five saved views, and an **automatically generated Trip Brief inside every trip**. The whole workspace was built with the Notion API (Python), so it can be recreated with one command.

Design choices (the optional note)

> **The founder should never have to learn anything.** Every decision follows from one rule: the founder opens the link and finds what they need in one or two taps, without scrolling through data meant for the assistant.
>
> 1. **Two taps to anything.** The top of the page has one line of guidance and a box that always shows the trip in progress and the next trip. Every trip link there, and the **Open the Trip Brief** link next to each trip, opens the trip already scrolled to its brief. The box updates itself.
> 2. **The brief reads in the order a traveller asks questions.** Where am I going, when, and where do I sleep (with a map link). Is there anything I must do (a visa problem appears in red). What is on today (shown while the trip is in progress). Who do I call. Then the plan for each day, in time order. The remaining details (flights, transport, booking numbers, documents) come last, in one short list.
> 3. **Only what is needed is shown.** The default view, **Upcoming trips**, is a simple table: trip, an *Open the Trip Brief* link, dates and countdown, soonest first. The Calendar, Timeline and By status views show just as little. The raw fields the assistant types into stay on the trip page for editing, but the founder's links skip past them.
> 4. **Always up to date, with no effort.** The assistant fills in plain fields only, with one line per itinerary event. A small script (GitHub Actions, started every 15 minutes, with a Cloudflare trigger as backup) rewrites the brief whenever a trip changes, and once a day during a trip to keep "Today" current. A static Notion template would go stale after the first edit. Lines typed in the wrong format are shown, never lost.
> 5. **Made for a phone, easy on the eyes.** Briefs use only headings, bullets and coloured boxes, with thin grey lines between sections and between days, so the eye can jump straight to the part it needs. There are no wide tables, so nothing needs sideways scrolling.

Requirements, as written in the brief

Brief	How it's met
✓ Single Trips database, one record per trip	One **Trips** database; every trip is one record.
✓ Trip name, who is traveling, date range	**Trip**, **Traveler**, **Travel Dates** (range), plus Status, Destination, Purpose and a **Countdown** ("15 days to go", "Travelling now").
✓ Flights, hotels, addresses, confirmations, meetings, notes	Flights (with booking codes), Hotel, Hotel Address, Hotel Confirmation, Ground Transport, Visa / Entry and Visa Notes, an **Itinerary** with one line per event (meetings, transfers, dinners), and Notes.
✓ Trip Brief: auto-generated, shareable	A **Trip Brief** at the top of every trip page, written automatically from the trip's fields and refreshed within 15 minutes of any edit. Shareable through the published link.
✓ Chronological timeline of key events	**Day by day**: every event sorted by date and time and grouped by day, with its confirmation number. While travelling, a **Today** box shows the current day first.

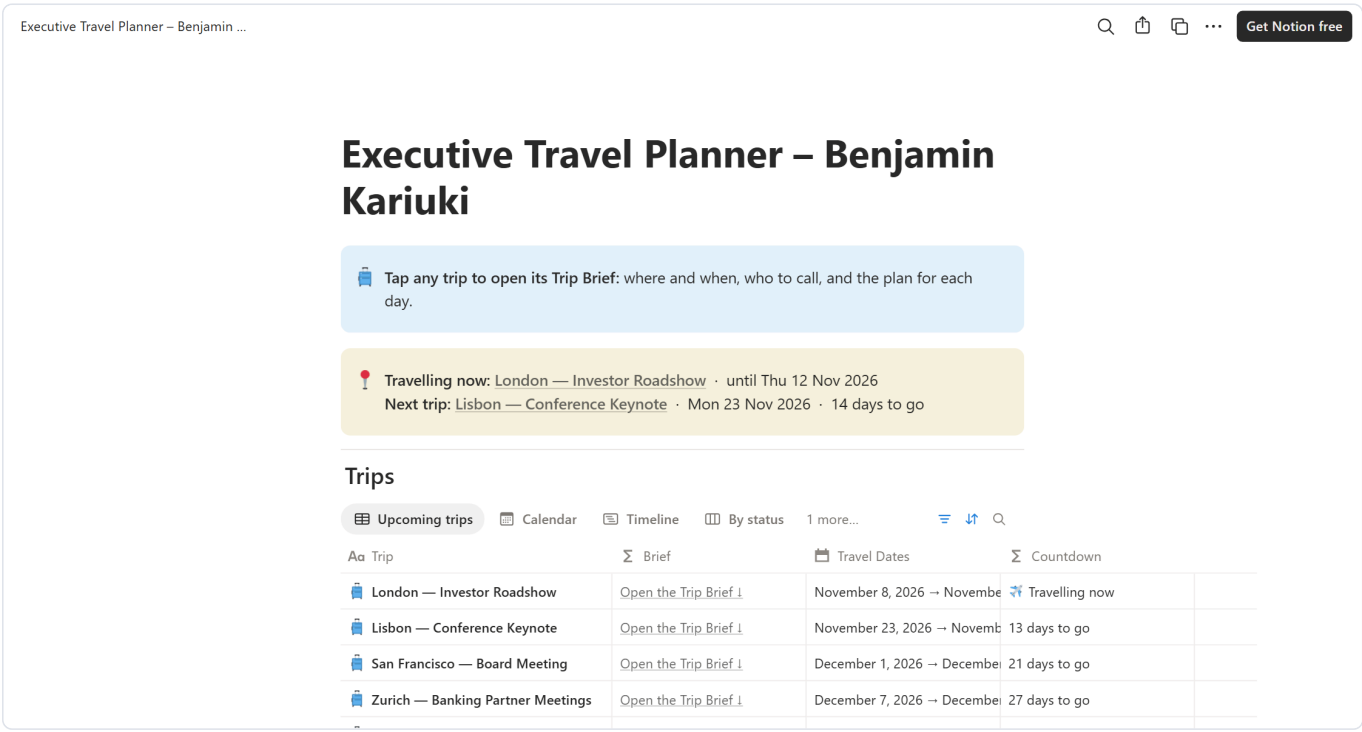
✓ Space to attach documents (tickets, visas)	The **Documents** field (sample e-ticket, hotel confirmation and insurance PDFs attached), listed in the brief, plus an **Extra files & notes** area that is never overwritten.
✓ Emergency / primary contact info	A red box near the top of every brief: primary contact (the EA), emergency contact and hotel.
✓ Timeline/calendar view of upcoming trips	**Upcoming trips** (the default), **Calendar** and **Timeline**; Done trips are hidden from Upcoming and Timeline.
✓ Overview grouped by status	A **By status** board with columns Planned / In-Progress / Done.
✓ Share to web, titled "Executive Travel Planner – {Your Name}"	Published, view-only: "Executive Travel Planner – Benjamin Kariuki".

How to check it in 3 minutes

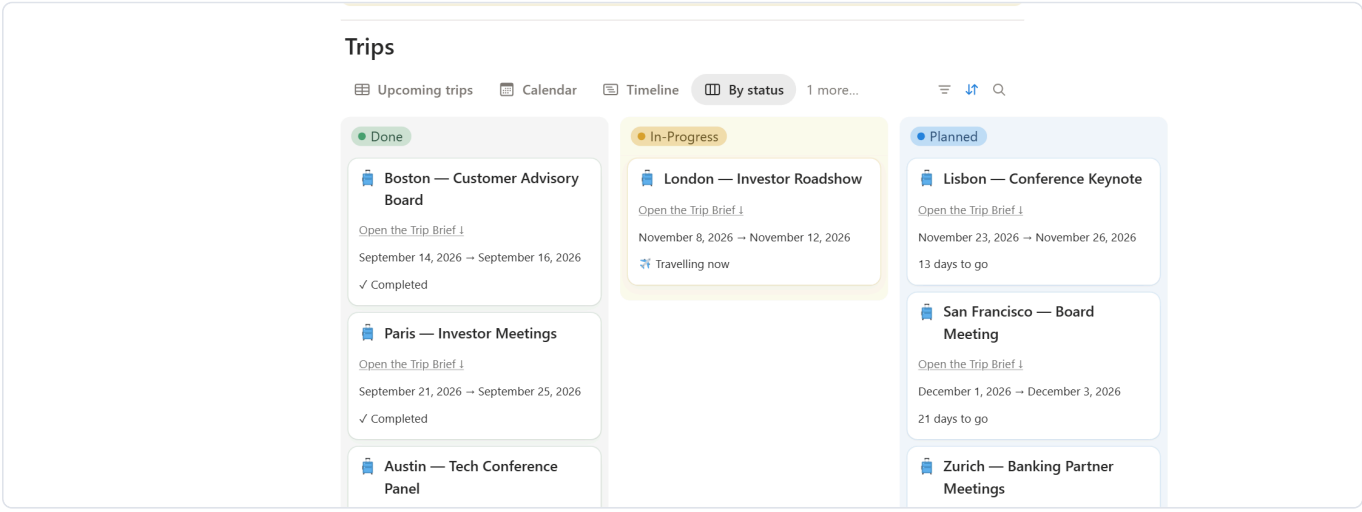
1. Open the Notion link. The yellow box at the top shows the trip in progress (*London – Investor Roadshow*) and the next trip.
2. Tap *London – Investor Roadshow* in that box (or *Open the Trip Brief* in the list). The trip opens already scrolled to its brief: where and where you sleep, Today, who to call, then each day.
3. Go back and switch between **Upcoming trips**, **Calendar**, **Timeline**, **By status** and **All trips**.
4. Open the link on a phone: the brief reads top to bottom with no sideways scrolling.

Testing

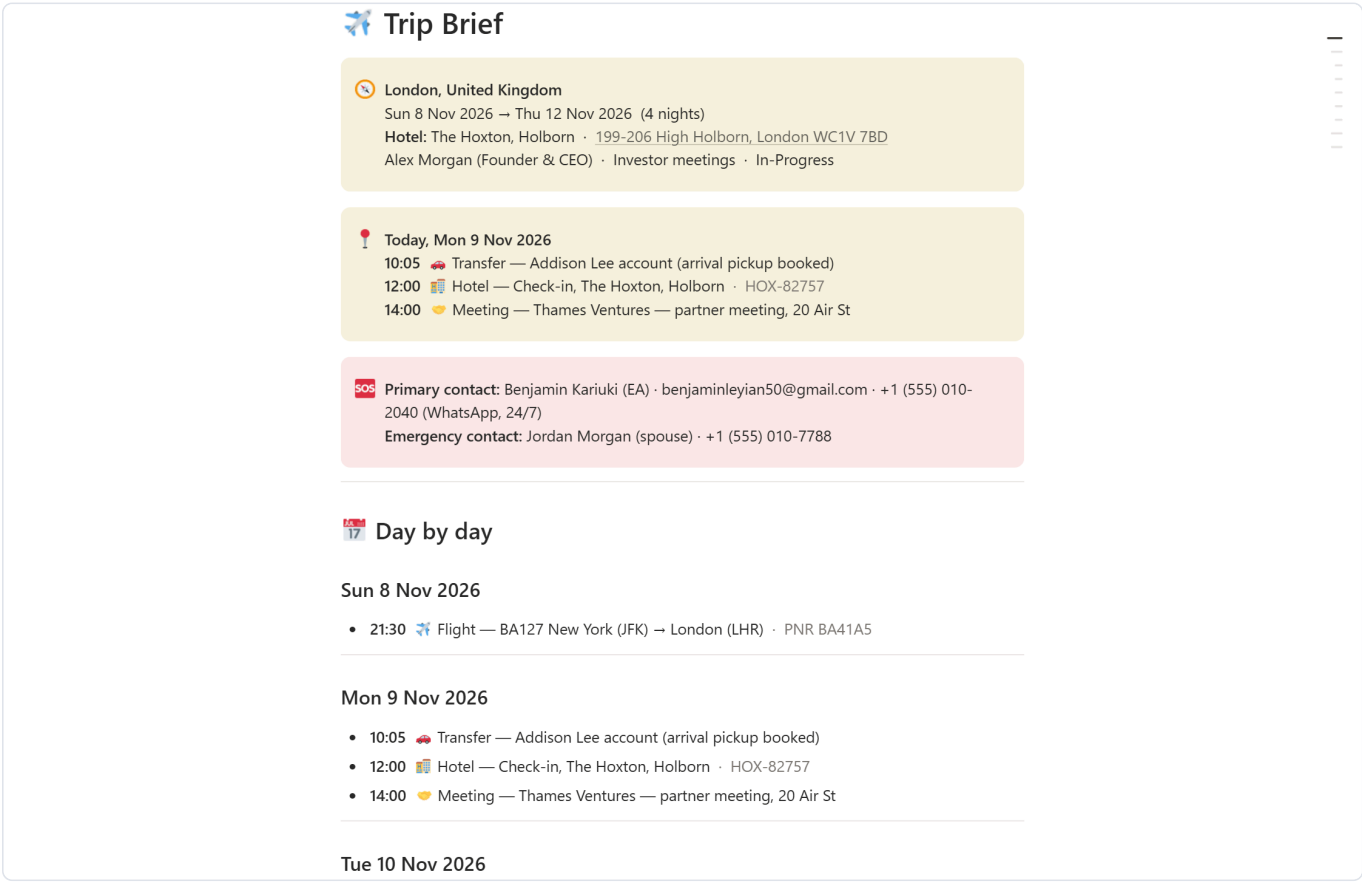
- 6 offline unit tests: itinerary sorting, brief order and contents, the Today and visa boxes, the refresh rules, the "Travelling now / Next trip" box and its direct-to-brief links. They run before every refresh.
- Live build: 23 trips with 23 complete briefs and 0 schedule conflicts. Editing one trip rebuilds only its brief, and notes under *Extra files & notes* survive a rebuild.



The planner page: one line of guidance, the automatic Travelling now / Next trip box, then the trips (Upcoming trips, the default view).



The By status view: Planned / In-Progress / Done.



A Trip Brief, generated automatically: where, when and where to sleep; Today; who to call; then each day.